

Lab 10 - Resource Requests, Limits, QoS and Autoscaling

Lab id	Lab 10
Day / Topic	Day 2 - Container Orchestration Fundamentals
Duration	50 minutes
Namespace	kcna-lab10
Mapping	LO2 *Identify the technical and practical requirements in a Kubernetes setup* A3 *Identify technical and practical requirements as well as stakeholders' demands* K3 *Objectives of solution architecture*
Cluster	Single-node kind cluster, Kubernetes v1.30+

1. Objective

By the end of this lab you will be able to:

1. State precisely what a request does (scheduling, and the denominator of HPA utilisation) and what a limit does (CPU throttling, memory OOMKill).
2. Prove all three QoS classes - Guaranteed, Burstable, BestEffort - from the live API with `kubectl get pod -o jsonpath='{.status.qosClass}'`, and explain the rule that produced each.
3. Cause and diagnose an OOMKill: `reason=OOMKilled`, exit code 137.
4. Apply a LimitRange (per-container defaults and bounds) and a ResourceQuota (aggregate namespace budget), and read the admission rejection each produces.
5. Author an autoscaling/v2 HorizontalPodAutoscaler and interpret its status honestly on a cluster with no metrics-server.

Read this before you start - the metrics-server caveat

>

A stock kind cluster does not run metrics-server. There is no `metrics.k8s.io` API, so:

>

* `kubectl top pod` fails with error: Metrics API not available, and

* your HPA will show `TARGETS: cpu: <unknown>/60%` with `ScalingActive=False`, `reason=FailedGetResourceMetric`.

>

That is the correct and expected result, and this lab treats it as the primary path. It is not a broken manifest and not a broken cluster - the measurement pipeline is simply absent. You will read the HPA's own conditions to prove that diagnosis rather than guess at it.

>

Section 7 gives the optional install if your instructor wants live metrics. It modifies `kube-system`, which is outside this lab's namespace, so it is instructor-run and explicitly opt-in.

2. Prerequisites

```
kubectl version --output=yaml | grep gitVersion | head -1
kubectl get nodes
```

Expected output:

```
gitVersion: v1.31.0
NAME          STATUS    ROLES          AGE   VERSION
kind-control-plane  Ready    control-plane  84m   v1.31.0
```

Check how much the single node actually has to give out - this is the budget every request in this lab is drawn against:

```
kubectl get node -o custom-columns=NODE:.metadata.name,CPU_ALLOC:.status.allocatable.cpu,
MEM_ALLOC:.status.allocatable.memory,PODS:.status.allocatable.pods
```

Expected output (values depend on how much you gave Docker Desktop):

```
NODE          CPU_ALLOC  MEM_ALLOC  PODS
kind-control-plane  8          8039488Ki  110
```

Confirm the metrics API really is absent, so the rest of the lab is grounded in fact:

```
kubectl get apiservice v1beta1.metrics.k8s.io
```

Expected output on a stock kind cluster:

```
Error from server (NotFound): apiservices.apiregistration.k8s.io "v1beta1.metrics.k8s.io" not found
```

```
kubectl top nodes
```

Expected output:

```
error: Metrics API not available
```

Write that down. It is the evidence for section 6b.

```
cd courseware/labs/lab-10-resources-requests-limits-autoscaling
ls manifests
```

Expected output:

```
00-namespace.yaml 31-resourcequota.yaml
10-pod-qos-guaranteed.yaml 40-deployment-quote-engine.yaml
11-pod-qos-burstable.yaml 50-hpa-quote-engine.yaml
12-pod-qos-besteffort.yaml 60-pod-quota-violation.yaml
20-pod-oomkill-ledger-compactor.yaml
30-limitrange.yaml
```

3. Scenario

Marina Freight Systems (MFS) has been running its staging namespace with no resource declarations at all. Last month a runaway ledger compaction consumed 6 GiB on a shared node and evicted the container-tracking API during the morning gate rush.

The platform lead's remediation plan is in data/workload-profiles.csv: every Harbour workload now has an agreed CPU/memory profile and a required QoS class. Finance additionally wants a hard ceiling on what the staging namespace can consume, so no single team can repeat the incident.

Your job:

1. Apply the profiles exactly as written and prove each workload landed in its required QoS class.
2. Reproduce the compaction incident safely, at 32 MiB instead of 6 GiB.
3. Install the namespace guardrails and show one of them rejecting an over-budget Pod.
4. Wire up an HPA for the rate-quote engine and report truthfully what it can and cannot do on this cluster.

4. Step-by-step procedure

Step 1 - Read the workload profiles

```
column -s, -t data/workload-profiles.csv
```

Expected output:

workload	manifest	cpu_request	cpu_limit	mem_request
mem_limit	expected_qos note			
berth-ledger	10-pod-qos-guaranteed.yaml	100m	100m	128Mi
128Mi	Guaranteed	requests equal limits for cpu and memory		
rate-quoter	11-pod-qos-burstable.yaml	50m	200m	64Mi
192Mi	Burstable	requests below limits so the container may burst		
notice-board	12-pod-qos-besteffect.yaml	none	none	none
none	BestEffort	no resources declared at all - apply before the LimitRange		
ledger-compactor	20-pod-oomkill-ledger-compactor.yaml	100m	100m	32Mi
32Mi	Guaranteed	memory limit deliberately below the working set - OOMKilled		
quote-engine	40-deployment-quote-engine.yaml	100m	300m	96Mi
256Mi	Burstable	HPA target - 60% utilisation of the 100m request is 60m		
cargo-modeller	60-pod-quota-violation.yaml	900m	1	128Mi
256Mi	Rejected	within the LimitRange max but over the namespace requests.cpu quota		

The expected_qos column is the contract. verification/checks.sh reads this file and compares every value against the live API - so if you edit the manifests, the checks will catch the drift.

Before applying anything, be able to state the rules:

QoS class	Rule
Guaranteed	*Every* container sets both cpu and memory, and for each, requests == limits
Burstable	At least one container sets a request or limit, but the Pod is not Guaranteed
BestEffort	*No* container sets any request or any limit

You never write qosClass. The API server derives it and writes it into .status.

Step 2 - Namespace and the three QoS Pods

```
kubectl apply -f manifests/00-namespace.yaml
```

Expected output:

```
namespace/kcna-lab10 created
```

Apply the three QoS Pods now, before the LimitRange. The order matters and Step 5 explains why.

```
kubectl apply -f manifests/10-pod-qos-guaranteed.yaml
kubectl apply -f manifests/11-pod-qos-burstable.yaml
kubectl apply -f manifests/12-pod-qos-besteffort.yaml
```

Expected output:

```
pod/berth-ledger created
pod/rate-quoter created
pod/notice-board created
```

```
kubectl -n kcna-lab10 wait --for=condition=Ready pod --all --timeout=90s
```

Expected output:

```
pod/berth-ledger condition met
pod/notice-board condition met
pod/rate-quoter condition met
```

Now prove the classes:

```
kubectl -n kcna-lab10 get pods \
-o custom-columns=POD:.metadata.name,QOS:.status.qosClass,CPU_REQ:.spec.containers[0].
resources.requests.cpu,CPU_LIM:.spec.containers[0].resources.limits.cpu,MEM_REQ:.spec.
containers[0].resources.requests.memory,MEM_LIM:.spec.containers[0].resources.limits.memory
```

Expected output:

POD	QOS	CPU_REQ	CPU_LIM	MEM_REQ	MEM_LIM
berth-ledger	Guaranteed	100m	100m	128Mi	128Mi
notice-board	BestEffort	<none>	<none>	<none>	<none>
rate-quoter	Burstable	50m	200m	64Mi	192Mi

Read each row against the rule table. berth-ledger has 100m == 100m and 128Mi == 128Mi → Guaranteed.

rate-quoter has requests strictly below limits → Burstable. notice-board declares nothing → BestEffort.

Single-Pod form, which is the one to memorise for the exam:

```
kubectl -n kcna-lab10 get pod berth-ledger -o jsonpath='{.status.qosClass}'; echo
```

Expected output:

```
Guaranteed
```

Confirm you cannot set it yourself - .status is not writable through apply:

```
grep -c 'qosClass' manifests/10-pod-qos-guaranteed.yaml || echo "qosClass appears nowhere in the manifest"
```

Expected output:

```
0
qosClass appears nowhere in the manifest
```

Why the class matters. Under node memory pressure the kubelet evicts in order: BestEffort first, then Burstable that exceed their requests, and Guaranteed last. notice-board is the first thing the cluster will sacrifice; berth-ledger is the last. That ordering is the whole reason MFS assigned classes per workload.

```
kubectl -n kcna-lab10 describe node "$(kubectl get nodes -o jsonpath='{.items[0].metadata.name}')" \
  | sed -n '/Non-terminated Pods/,/Allocated resources/p' | grep -E 'kcna-lab10|NAMESPACE'
```

Expected output:

NAMESPACE	NAME	CPU Requests	CPU Limits	Memory
kcna-lab10	berth-ledger	100m (1%)	100m (1%)	128Mi (1%)
kcna-lab10	notice-board	0 (0%)	0 (0%)	0 (0%)
kcna-lab10	rate-quoter	50m (0%)	200m (2%)	64Mi (0%)

notice-board books zero. The scheduler treats a BestEffort Pod as free, which is exactly how a node ends up oversubscribed.

Step 3 - Cause an OOMKill

Reproduce the MFS incident, scaled down to 32 MiB:

```
sed -n '/resources:/,/securityContext:/p' manifests/20-pod-oomkill-ledger-compactor.yaml
```

Expected output:

```
resources:
  requests:
    cpu: 100m
    memory: 32Mi
  limits:
    cpu: 100m
    memory: 32Mi
  securityContext:
```

The container will try to write a 64 MiB file into a memory-backed emptyDir, whose pages are charged to its own cgroup. It has a 32 MiB ceiling.

```
kubectl apply -f manifests/20-pod-oomkill-ledger-compactor.yaml
sleep 15
kubectl -n kcna-lab10 get pod ledger-compactor
```

Expected output:

```
pod/ledger-compactor created
NAME          READY   STATUS    RESTARTS   AGE
ledger-compactor 0/1     OOMKilled 0           15s
```

The evidence, from the container status rather than the summary column:

```
kubectl -n kcna-lab10 get pod ledger-compactor \
-o jsonpath='{reason={.status.containerStatuses[0].state.terminated.reason} exitCode={.status.containerStatuses[0].state.terminated.exitCode} limit={.spec.containers[0].resources.limits.memory}}'; echo
```

Expected output:

```
reason=OOMKilled exitCode=137 limit=32Mi
```

```
kubectl -n kcna-lab10 describe pod ledger-compactor | sed -n '/^ *State:/,/ ^ *Ready:/p'
```

Expected output:

```
State:      Terminated
Reason:     OOMKilled
Exit Code:  137
Started:    Sat, 05 Sep 2026 10:41:02 +0800
Finished:   Sat, 05 Sep 2026 10:41:04 +0800
Ready:      False
```

```
kubectl -n kcna-lab10 logs ledger-compactor
```

Expected output - the work never finished:

```
compactor: memory limit is 32Mi; attempting a 64 MiB working set
```

Diagnosis.

* 137 = 128 + 9. The container was killed by signal 9 (SIGKILL). That is the kernel OOM killer, not Kubernetes deciding to stop it.

* The COMPACTION-COMPLETE line never printed, so the process died mid-dd.

* Its QoS class is still Guaranteed - being Guaranteed does not exempt you from your own limit. Guaranteed protects you from *other* workloads causing your eviction; nothing protects you from exceeding a ceiling you declared yourself.

```
kubectl -n kcna-lab10 get pod ledger-compactor -o jsonpath='{.status.qosClass} / {.status.phase}'; echo
```

Expected output:

```
Guaranteed / Failed
```

Memory and CPU limits behave completely differently. Memory is incompressible: exceed the limit and you are killed. CPU is compressible: exceed the limit and you are **throttled** - the container runs slower but survives. Never treat the two as interchangeable.

Step 4 - Set namespace guardrails

```
kubectl apply -f manifests/30-limitrange.yaml
kubectl apply -f manifests/31-resourcequota.yaml
```

Expected output:

```
limitrange/mfs-container-limits created
resourcequota/mfs-lab-quota created
```

```
kubectl -n kcna-lab10 describe limitrange mfs-container-limits
```

Expected output:

```
Name:      mfs-container-limits
Namespace: kcna-lab10
Type       Resource  Min   Max   Default Request  Default Limit  Max Limit/Request Ratio
-----
Container  cpu        10m   1     50m
Container  memory     16Mi  512Mi 64Mi  256Mi  -
```

```
kubectl -n kcna-lab10 describe resourcequota mfs-lab-quota
```

Expected output:

```
Name:      mfs-lab-quota
Namespace: kcna-lab10
Resource   Used   Hard
-----
count/deployments.apps  0      5
limits.cpu              300m   3
limits.memory           320Mi  2Gi
pods                   3      15
requests.cpu            150m   1
requests.memory         192Mi  1Gi
```

ledger-compactor is in a terminal phase (Failed), so the quota controller no longer counts it. notice-board counts as 3 in pods but 0 in every compute resource - a BestEffort Pod consumes quota **slots**, not quota **capacity**.

Step 5 - Watch the LimitRange rewrite a Pod

This is why the ordering in Step 2 mattered. Create a Pod that declares nothing - the same shape as notice-board:

```
kubectl -n kcna-lab10 run late-arrival --image=registry.k8s.io/pause:3.9 --restart=Never
sleep 5
kubectl -n kcna-lab10 get pod late-arrival \
-o custom-columns=POD:.metadata.name,QOS:.status.qosClass,CPU_REQ:.spec.containers[0].
resources.requests.cpu,CPU_LIM:.spec.containers[0].resources.limits.cpu,MEM_REQ:.spec.
containers[0].resources.requests.memory,MEM_LIM:.spec.containers[0].resources.limits.memory
```

Expected output:

```
pod/late-arrival created
POD          QOS      CPU_REQ  CPU_LIM  MEM_REQ  MEM_LIM
late-arrival  Burstable 50m      500m     64Mi     256Mi
```

It is Burstable, not BestEffort, and the manifest never said so. The LimitRange admission plugin injected defaultRequest and default into the Pod spec before it was persisted. Compare with notice-board, created *before* the LimitRange existed:

```
kubectl -n kcna-lab10 get pods -l qos-demo=besteffort -o jsonpath='{.items[0].metadata.name}={.items[0].status.qosClass}'; echo
kubectl -n kcna-lab10 get pod late-arrival -o jsonpath='late-arrival={.status.qosClass}'; echo
```

Expected output:

```
notice-board=BestEffort
late-arrival=Burstable
```

Once a LimitRange with defaults exists, BestEffort is impossible in that namespace. For most platform teams that is the point: it makes "declared nothing" unrepresentable. Note also that the LimitRange applied retroactively to **nothing** - notice-board was untouched. Admission plugins only see creates and updates.

Remove the temporary Pod:

```
kubectl -n kcna-lab10 delete pod late-arrival
```

Expected output:

```
pod "late-arrival" deleted
```

Step 6 - Deploy the HPA target

```
kubectl apply -f manifests/40-deployment-quote-engine.yaml
kubectl -n kcna-lab10 rollout status deploy/quote-engine --timeout=120s
```

Expected output:

```
deployment.apps/quote-engine created
Waiting for deployment "quote-engine" rollout to finish: 0 of 2 updated replicas are available.
..
deployment "quote-engine" successfully rolled out
```

```
kubectl -n kcna-lab10 get pods -l app=quote-engine \
-o custom-columns=POD:.metadata.name,QOS:.status.qosClass,CPU_REQ:.spec.containers[0].resources.requests.cpu,CPU_LIM:.spec.containers[0].resources.limits.cpu
```

Expected output:

```
POD          QOS      CPU_REQ  CPU_LIM
quote-engine-6b7f9c8d54-kx2mp  Burstable 100m     300m
quote-engine-6b7f9c8d54-w9tqz  Burstable 100m     300m
```

The quota now reflects the Deployment:

```
kubectl -n kcna-lab10 get resourcequota mfs-lab-quota \
-o jsonpath='requests.cpu used={.status.used.requests\.cpu} of {.status.hard.requests\.cpu}';
echo
```

Expected output:

```
requests.cpu used=350m of 1
```

100m (berth-ledger) + 50m (rate-quoter) + 0 (notice-board) + 200m (quote-engine ×2) = 350m. Remember that figure - Step 6 of the failure injection uses it.

Step 7 - Create the HPA and read it honestly

```
kubectl apply -f manifests/50-hpa-quote-engine.yaml
```

Expected output:

```
horizontalpodautoscaler.autoscaling/quote-engine created
```

```
sleep 20
kubectl -n kcna-lab10 get hpa quote-engine
```

Expected output on a stock kind cluster (this is the honest result - do not expect numbers here):

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS
quote-engine	Deployment/quote-engine	cpu: <unknown>/60%, memory: <unknown>/200Mi	2	
6	20s			

Do not guess why. Ask the object:

```
kubectl -n kcna-lab10 get hpa quote-engine \
-o jsonpath='{range .status.conditions[*]}{.type}={.status} reason={.reason}{"\n"}{end}'
```

Expected output:

```
AbleToScale=True reason=SucceededGetScale
ScalingActive=False reason=FailedGetResourceMetric
```

```
kubectl -n kcna-lab10 describe hpa quote-engine | sed -n '/^Conditions:$/, $p'
```

Real output:

Conditions:

Type	Status	Reason	Message
-----	-----	-----	-----
AbleToScale	True	SucceededGetScale	the HPA controller was able to get the target's current scale
ScalingActive	False	FailedGetResourceMetric	the HPA was unable to compute the replica count: failed to get cpu utilization: unable to get metrics for resource cpu: unable to fetch metrics from resource metrics API: the server could not find the requested resource (get pods.metrics.k8s.io)

Events:

Type	Reason	Age	From	Message
----	-----	----	----	-----
Warning	FailedGetResourceMetric	5s (x3 over 35s)	horizontal-pod-autoscaler	failed to get cpu utilization: unable to get metrics for resource cpu: unable to fetch metrics from resource metrics API: the server could not find the requested resource (get pods.metrics.k8s.io)
Warning	FailedComputeMetricsReplicas	5s (x3 over 35s)	horizontal-pod-autoscaler	the server could not find the requested resource (get pods.metrics.k8s.io)

Read the condition pair, it is a complete diagnosis:

* AbleToScale=True - the HPA found the Deployment and *could* scale it. The scaleTargetRef is correct.

* ScalingActive=False, reason=FailedGetResourceMetric - it has no numbers to scale on. The message names the exact missing API: pods.metrics.k8s.io.

That API is served by metrics-server, which kind does not install. Confirm once more, from the API registry rather than from inference:

```
kubectl get apiservice | grep -c metrics.k8s.io || echo "no metrics.k8s.io APIService is registered"
kubectl -n kcna-lab10 top pod
```

Expected output:

```
0
no metrics.k8s.io APIService is registered
error: Metrics API not available
```

What the HPA would do if metrics existed. The arithmetic is worth knowing even without a live pipeline:

```
desired = ceil( currentReplicas x ( currentMetricValue / desiredMetricValue ) )
```

With averageUtilization: 60 against a 100m request, the target is 60m of CPU per Pod. If the two Pods averaged 120m:

```
desired = ceil( 2 x (120 / 60) ) = 4
```

bounded by minReplicas: 2 / maxReplicas: 6, and damped by the behavior block: at most 2 Pods added per 60 s after a 30 s stabilisation window, and at most 50 % removed per 60 s after a 300 s window. Read those values off the object:

```
kubectl -n kcna-lab10 get hpa quote-engine \
-o jsonpath='{target={.spec.metrics[0].resource.target.averageUtilization}% min={.spec.minReplicas} max={.spec.maxReplicas} scaleUpStabilisation={.spec.behavior.scaleUp.stabilizationWindowSeconds}s scaleDownStabilisation={.spec.behavior.scaleDown.stabilizationWindowSeconds}s}'; echo
```

Expected output:

```
target=60% min=2 max=6 scaleUpStabilisation=30s scaleDownStabilisation=300s
```

Notice what this proves about requests: an HPA on Utilization is meaningless without a CPU request, because the request is the denominator. A BestEffort Pod cannot be autoscaled on utilisation at all.

Optional (instructor-run) - install metrics-server so the HPA reports real numbers

This step writes to the kube-system namespace, outside this lab's namespace, and is therefore opt-in. Skip it unless your instructor asks for it; every assertion in verification/checks.sh passes either way.

>

kind nodes serve the kubelet API with a self-signed certificate that metrics-server will not trust by default, so the --kubelet-insecure-tls argument is mandatory here. It is not appropriate for a production cluster.

>

```
```bash
```

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/download/v0.7.2/components.yaml
```

```
kubectl -n kube-system patch deployment metrics-server --type=json \
```

```
-p '[{"op": "add", "path": "/spec/template/spec/containers/0/args/-", "value": "--kubelet-insecure-tls"}]'
```

```
kubectl -n kube-system rollout status deploy/metrics-server --timeout=180s
```

```
```
```

>

Expected output:

>

```
```
```

```
serviceaccount/metrics-server created
```

```
clusterrole.rbac.authorization.k8s.io/system:aggregated-metrics-reader created
```

```
...
```

```
apiservice.apiregistration.k8s.io/v1beta1.metrics.k8s.io created
```

```
deployment.apps/metrics-server patched
```

```
deployment "metrics-server" successfully rolled out
```

```
```
```

>

Then, after roughly 60 seconds of scrape warm-up:

>

```
```bash
```

```
kubectl -n kcna-lab10 top pod
```

```
kubectl -n kcna-lab10 get hpa quote-engine
```

```
```
```

>

Expected output - now with real figures, which will differ on your machine:

>

...

| NAME | CPU(cores) | MEMORY(bytes) |
|-------------------------------|------------|---------------|
| berth-ledger | 1m | 4Mi |
| notice-board | 0m | 0Mi |
| quote-engine-6b7f9c8d54-kx2mp | 1m | 7Mi |
| quote-engine-6b7f9c8d54-w9tqz | 1m | 7Mi |
| rate-quoter | 1m | 4Mi |

| NAME | REFERENCE TARGETS | MINPODS | MAXPODS | REPLICAS | AGE |
|--------------------------------------|--------------------------------|---------|---------|----------|-----|
| quote-engine Deployment/quote-engine | cpu: 1%/60%, memory: 7Mi/200Mi | 2 | 6 | 2 | 6m |

...

>

The Pods are idle, so utilisation sits far below the 60 % target and the HPA holds at minReplicas: 2. To remove metrics-server afterwards, delete exactly what you installed: `kubectl delete -f https://github.com/kubernetes-sigs/metrics-server/releases/download/v0.7.2/components.yaml`.

5. Verification

```
bash verification/checks.sh; echo "exit=$?"
```

The script reads data/workload-profiles.csv and asserts that each workload's live .status.qosClass equals the expected_qos column. It also checks the OOMKill evidence, the LimitRange values, the ResourceQuota, and the HPA spec.

For the metrics pipeline it branches on reality: it queries the v1beta1.metrics.k8s.io APIService, and passes either when metrics-server is absent and the HPA correctly reports FailedGetResourceMetric, or when metrics-server is present and the HPA is scaling. It never asserts that `kubectl top` worked.

Run it after section 6 - the cargo-modeller rejection check requires that the failure injection has been attempted. Full transcript: [verification/expected-output.md](#).

6. Failure injection

6a. A Pod that breaches the namespace quota

Finance's ceiling is requests.cpu: 1 for the whole namespace. The analytics team asks for 900m on top of the 350m already committed.

```
kubectl apply -f manifests/60-pod-quota-violation.yaml
```

Real error text:

```
Error from server (Forbidden): error when creating "manifests/60-pod-quota-violation.yaml":
pods "cargo-modeller" is forbidden: exceeded quota: mfs-lab-quota, requested: requests.
cpu=900m, used: requests.cpu=350m, limited: requests.cpu=1
```

Confirm nothing was created - this is an admission rejection, so there is no Pending Pod to inspect:

```
kubectl -n kcna-lab10 get pod cargo-modeller
```

Expected output:

```
Error from server (NotFound): pods "cargo-modeller" not found
```

Diagnosis. The message is fully self-describing: requested + used > limited, i.e. 900m + 350m > 1000m. Note what did not happen:

* It was not rejected by the LimitRange - 900m is within the per-container max of 1. Confirm the distinction:

```
kubectl -n kcna-lab10 get limitrange mfs-container-limits -o jsonpath='per-container max cpu={.
spec.limits[0].max.cpu}'; echo
kubectl -n kcna-lab10 get resourcequota mfs-lab-quota -o jsonpath='namespace-wide requests.
cpu={.status.hard.requests.cpu} used={.status.used.requests.cpu}'; echo
```

Expected output:

```
per-container max cpu=1
namespace-wide requests.cpu=1 used=350m
```

* It never reached the scheduler. Compare this with Lab 09, where an unschedulable Pod was created and sat Pending. Admission rejects; scheduling defers. Different failure, different evidence, different fix.

Fix - pick one and justify it to Finance:

```
sed 's/cpu: 900m/cpu: 400m/' manifests/60-pod-quota-violation.yaml | kubectl apply -f -
kubectl -n kcna-lab10 get pod cargo-modeller -o jsonpath='{.metadata.name} qos={.status.
qosClass} cpuRequest={.spec.containers[0].resources.requests.cpu}'; echo
```

Expected output:

```
pod/cargo-modeller created
cargo-modeller qos=Burstable cpuRequest=400m
```

The alternative is to raise the quota (`kubectl -n kcna-lab10 patch resourcequota mfs-lab-quota -p '{"spec":{"hard":{"requests.cpu":"2"}}}'`) - but that is a budget decision, not an engineering one, which is precisely why the quota exists.

Restore the rejected state before verifying:

```
kubectl -n kcna-lab10 delete pod cargo-modeller
```

Expected output:

```
pod "cargo-modeller" deleted
```

6b. A Pod that breaches the LimitRange

A different admission plugin, a different message:

```
kubectl -n kcna-lab10 run oversized --image=registry.k8s.io/pause:3.9 --restart=Never \
  --overrides='{ "spec": { "containers": [ { "name": "oversized", "image": "registry.k8s.io/pause:3.9",
    "resources": { "requests": { "cpu": "50m", "memory": "64Mi" }, "limits": { "cpu": "2", "memory":
    "128Mi" } } } ] } }'
```

Real error text:

```
Error from server (Forbidden): pods "oversized" is forbidden: maximum cpu usage per Container
is 1, but limit is 2
```

Diagnosis. This one names a per-container bound (maximum cpu usage per Container), not a namespace total. LimitRange polices the shape of each container; ResourceQuota polices the sum across the namespace. Both are admission plugins, both reject before persistence, and a Pod must satisfy both.

6c. The HPA that cannot scale

Already produced in Step 7 and worth restating as a failure to recognise on sight:

```
ScalingActive    False    FailedGetResourceMetric ... unable to fetch metrics from resource
metrics API: the server could not find the requested resource (get pods.metrics.k8s.io)
```

Diagnosis. AbleToScale=True with ScalingActive=False means *the HPA is wired correctly and blind*. The fix is never in the HPA manifest - it is to install metrics-server (or a custom-metrics adapter for type: Pods/type: External metrics). If instead you see ScalingActive=False with reason=InvalidSelector or AbleToScale=False with FailedGetScale, the fault is in the manifest: the scaleTargetRef names an object that does not exist.

7. Troubleshooting

| Symptom | Cause | Fix |
|--|---|--|
| STATUS: OOMKilled, Exit Code: 137 | The container's working set exceeded resources.limits.memory; the kernel OOM killer sent SIGKILL | Raise the memory limit to the real working set, or reduce what the process allocates. Memory is incompressible - there is no throttling fallback |
| Container runs but is inexplicably slow, no restarts | CPU limit throttling. CPU is compressible, so exceeding the limit slows the container instead of killing it | Raise limits.cpu, or remove the CPU limit and rely on the request for scheduling |
| pods "x" is forbidden: exceeded quota: <name>, requested: ..., used: ..., limited: ... | Namespace ResourceQuota breached in aggregate | Lower the Pod's requests, delete unused Pods, or have the quota raised |
| pods "x" is forbidden: maximum cpu usage per Container is 1, but limit is 2 | Per-container LimitRange max breached | Bring the container within min/max, or change the LimitRange |
| pods "x" is forbidden: failed quota: <name>: must specify limits.cpu | A ResourceQuota names a compute resource, so every new Pod must declare it | Declare the resource on the container, or add a LimitRange default/defaultRequest so it is injected automatically |
| A Pod that declared nothing is Burstable, not BestEffort | A LimitRange with default/defaultRequest injected values at admission | Expected behaviour. BestEffort cannot exist in a namespace with LimitRange defaults |
| error: Metrics API not available from kubectl top | metrics-server is not installed - the stock kind default | Install metrics-server with --kubelet-insecure-tls (see the optional step), or accept that top and utilisation-based HPAs are unavailable |
| HPA TARGETS shows <unknown>/60% | No metrics source; the HPA cannot compute a ratio | Check ScalingActive - FailedGetResourceMetric means missing metrics-server, not a bad manifest |
| HPA never scales even with metrics, replicas stuck at minReplicas | Utilisation is genuinely below target, or the scaleUp stabilisation window has not elapsed | kubectl describe hpa and read the events; check .spec.behavior.scaleUp.stabilizationWindowSeconds |
| HPA AbleToScale=False
reason=FailedGetScale | scaleTargetRef points at an object that does not exist or is a kind without a scale subresource | Correct scaleTargetRef.kind/name; DaemonSets have no scale subresource and cannot be an HPA target |

8. Cleanup

Delete only this lab's namespace. The LimitRange, ResourceQuota, HPA, Deployment and all Pods are namespaced objects and go with it.

```
kubectl delete namespace kcna-lab10
```

Expected output:

```
namespace "kcna-lab10" deleted
```

```
kubectl get ns kcna-lab10
```

Expected output:

```
Error from server (NotFound): namespaces "kcna-lab10" not found
```

If you took the optional metrics-server step, remove exactly what you installed:

```
kubectl delete -f https://github.com/kubernetes-sigs/metrics-server/releases/download/v0.7.2/components.yaml
```

9. What you learned

- * Requests are a scheduling reservation and the denominator of HPA utilisation. Limits are an enforcement ceiling. They are different fields answering different questions, and a workload profile must set both deliberately.
- * CPU is compressible, memory is not. Exceed a CPU limit and you are throttled; exceed a memory limit and you are OOMKilled with exit code 137 (128 + SIGKILL).
- * The three QoS classes are *derived* by the API server, never declared: Guaranteed (every container sets cpu and memory with requests == limits), Burstable (something set, not Guaranteed), BestEffort (nothing set). Read it with `kubectl get pod -o jsonpath='{.status.qosClass}'`.
- * QoS drives eviction order under node pressure: BestEffort first, Guaranteed last. It does not protect you from your own limit.
- * A LimitRange shapes each container at admission - defaultRequest/default inject values, min/max reject outliers. Its side effect is that BestEffort becomes impossible in that namespace.
- * A ResourceQuota is an aggregate namespace budget, and naming a compute resource in it makes that resource mandatory on every new Pod.
- * Both are admission controls: a rejected Pod is never created, which is a different failure mode from Lab 09's Pending Pod that was created but could not be placed.
- * An autoscaling/v2 HPA needs a metrics source. On a stock kind cluster there is none, so the honest reading is TARGETS: <unknown> with AbleToScale=True / ScalingActive=False, reason=FailedGetResourceMetric. That condition pair - wired correctly, but blind - is the signature to recognise.

10. Further reading

- * Resource Management for Pods and Containers - <<https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/>>
- * Pod Quality of Service Classes - <<https://kubernetes.io/docs/concepts/workloads/pods/pod-qos/>>
- * Assign Memory Resources / troubleshoot OOMKill - <<https://kubernetes.io/docs/tasks/configure-pod-container/assign-memory-resource/>>
- * Limit Ranges - <<https://kubernetes.io/docs/concepts/policy/limit-range/>>
- * Resource Quotas - <<https://kubernetes.io/docs/concepts/policy/resource-quotas/>>
- * Horizontal Pod Autoscaling - <<https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>>
- * HPA walkthrough (includes the metrics-server requirement) - <<https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale-walkthrough/>>
- * Resource Metrics Pipeline - <<https://kubernetes.io/docs/tasks/debug/debug-cluster/resource-metrics-pipeline/>>
- * Node-pressure Eviction - <<https://kubernetes.io/docs/concepts/scheduling-eviction/node-pressure-eviction/>>